SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1703**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

IE.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. **Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty**

ii. **Explain how A* would handle the same situation, including how it balances cost and heuristic**

iii. **Analyze the impact of heuristic accuracy on both algorithms**

iv. **Discuss how dynamic changes (blocked paths) affect search performance**

v. **Recommend the most suitable algorithm and justify your decision considering realtime constraints, optimality, and safety**

ANSWER:
Scenario Understanding

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic because roads and routes may become blocked due to floods, and weather conditions such as heavy rain and strong winds increase travel time and risk. The drone receives partial real-time information about blocked paths and uses a heuristic estimate (straight-line distance) to estimate the distance to the destination. Since medicines must be delivered quickly, the drone must find a safe and efficient route with minimum delay.

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
How Greedy Best-First Search Works

Greedy Best-First Search (GBFS) chooses the next path based only on the heuristic value **h(n)**, which estimates the remaining distance from the current location to the destination.

Evaluation Function:

f(n) = h(n)

The algorithm always selects the node that appears closest to the goal without considering the actual travel cost already incurred.

Behavior in the Scenario

- The drone calculates the straight-line distance from its current position to the destination.
- It always chooses the path with the smallest heuristic value.

- If a route becomes blocked due to flooding, the drone abandons that path and searches for another nearby route.

Strengths

- Finds a route very quickly.
- Requires less computation.
- Suitable when immediate decisions are required.
- Useful when heuristic estimates are reasonably accurate.

Weaknesses

- Does not guarantee the shortest or safest path.
- Ignores movement costs caused by weather or terrain.
- May enter dead ends if blocked paths are discovered later.
- Often requires frequent replanning in dynamic environments.
- Can produce unsafe routes during emergencies.

Example

Suppose two possible routes are available:

- Route A: Straight distance = 4 km, Flooded area, Travel cost = 20
- Route B: Straight distance = 6 km, Clear path, Travel cost = 10

Greedy Best-First Search selects **Route A** because it appears closer to the destination, even though it is slower and riskier.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

How A* Search Works

A* Search considers both:

- Actual cost from the starting point **g(n)**
- Estimated remaining distance **h(n)**

Evaluation Function:

f(n) = g(n) + h(n)

The algorithm balances the distance already travelled with the estimated distance remaining.

Behavior in the Scenario

The drone continuously evaluates:

- Distance travelled
- Weather conditions
- Terrain difficulty
- Flooded roads
- Remaining distance

Whenever new information is received, the drone recalculates the path and chooses the safest and least costly route.

Advantages

- Finds the optimal path if the heuristic is admissible.
- Considers both travel cost and destination distance.
- Avoids dangerous or expensive routes.
- Produces more reliable decisions.

Disadvantages

- Requires more computation than Greedy Search.
- Uses more memory.
- Slightly slower because more nodes are evaluated.

Example

Suppose:

Route	g(n)	h(n)	f(n)=g+h
A	8	4	12
B	5	7	12
C	6	3	9

A* chooses **Route C** because it has the smallest total estimated cost (9), even though it is not the closest route.

iii. Analyze the impact of heuristic accuracy on both algorithms

The quality of the heuristic function greatly affects the performance of both algorithms.

Greedy Best-First Search
Accurate Heuristic

- Quickly reaches the destination.
- Explores fewer unnecessary paths.
- Faster search.

Inaccurate Heuristic

- Selects poor routes.
- May repeatedly encounter blocked roads.
- Can travel longer distances.
- Performance decreases significantly.

A* Search
Accurate Heuristic

- Finds the optimal path quickly.
- Expands fewer nodes.
- Reduces computation time.

Inaccurate Heuristic

- Explores more nodes before reaching the goal.
- Takes longer to compute.
- Still considers actual path cost, so it usually finds a better route than Greedy Search.

Comparison

Factor	Greedy Best-First Search	A* Search
Depends on heuristic	Very High	High
Sensitive to errors	Very High	Moderate
Route quality	Often poor	Usually optimal
Search efficiency	High with good heuristic	Good with accurate heuristic

iv. Discuss how dynamic changes (blocked paths) affect search performance

In a flood-affected region, conditions change continuously.

Examples:

- Flood water blocks roads.
- Bridges collapse.
- Heavy rain increases travel time.
- Wind changes drone movement cost.

Effect on Greedy Best-First Search

- Frequently chooses blocked paths because only heuristic distance is considered.
- Must restart or replan often.
- Wastes time exploring dead ends.
- Delivery may be delayed.

Effect on A* Search

- Recalculates path using updated travel costs.
- Avoids blocked or high-cost routes.
- Produces safer and more efficient paths.
- Better adapts to changing environments.

Although A* requires more computation, it handles dynamic environments much more effectively.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety
Recommended Algorithm: A* Search

A* Search is the most suitable algorithm for emergency medicine delivery because it balances speed, optimality, and safety.

Justification

1. Better Decision Making

A* considers both the distance travelled and the estimated distance remaining. This enables the drone to choose routes that minimize overall travel cost rather than simply moving toward the closest destination.

2. Handles Dynamic Conditions

When flooded roads or adverse weather are detected, A* updates its path based on the latest information. This reduces the chance of delays and failed deliveries.

3. Improved Safety

By accounting for terrain difficulty, weather, and movement cost, A* avoids risky routes and selects safer alternatives, which is essential during disaster relief operations.

4. Optimal Route Selection

If the heuristic is admissible, A* guarantees the least-cost path. This helps ensure medicines are delivered efficiently while conserving battery power and reducing travel time.

5. Suitable for Emergency Operations

Although Greedy Best-First Search is faster, it may choose unsafe or blocked paths because it ignores actual travel costs. In contrast, A* provides more reliable and cost-effective routing, making it better suited for emergency medical deliveries where both speed and safety are critical.

Conclusion

In this flood-affected emergency scenario, both Greedy Best-First Search and A* Search use heuristic information to guide the drone. However, Greedy Best-First Search focuses only on the estimated distance, making it fast but less reliable in uncertain environments. A* Search combines the actual travel cost with the heuristic estimate, enabling it to find safer and more efficient routes. Considering the dynamic environment, blocked paths, weather conditions, and the need for reliable medicine delivery, **A* Search is the most appropriate algorithm because it provides the best balance between real-time performance, optimality, and safety.**

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. **Explain how Hill Climbing would work in this scenario and identify its limitations**

ii. **Discuss issues like local maxima, plateaus, and ridges with real-world interpretation**

iii. **Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations**

iv. **Recommend the best approach and justify your choice based on scalability and adaptability**

ANSWER:

Scenario Understanding

A smart city uses an AI system to optimize traffic signal timings across hundreds of intersections. The objectives are to minimize total waiting time, reduce fuel consumption, and decrease traffic congestion. Since traffic conditions change continuously and the number of possible signal timing combinations is extremely large, finding the best solution is a complex optimization problem. The AI system must continuously adjust signal timings while avoiding poor local solutions.

Problem Formulation

This problem can be formulated as an **Optimization Problem**, where the goal is to find the best traffic signal timings that minimize waiting time, fuel consumption, and congestion while satisfying traffic constraints.

Objective Function

The AI system aims to minimize:

- Total vehicle waiting time
- Fuel consumption caused by idling vehicles
- Traffic congestion at intersections

Decision Variables

- Green signal duration
- Red signal duration
- Yellow signal duration
- Signal sequence at each intersection
- Coordination between nearby intersections

Constraints

- Minimum and maximum green light duration.
- Safe transition between traffic signals.
- Priority for emergency vehicles.
- Pedestrian crossing time.
- Coordination between neighboring intersections.

Why Traditional Search is Inefficient

Traditional search algorithms such as Breadth-First Search (BFS) or Depth-First Search (DFS) are not suitable because:

- The search space is extremely large due to hundreds of intersections and numerous timing combinations.
- Traffic conditions change continuously, requiring frequent updates.
- Exhaustively exploring every possible solution would consume excessive time and memory.
- Real-time decision-making is impossible using traditional search methods.

Therefore, optimization algorithms are more appropriate for this scenario.

i. Explain how Hill Climbing would work in this scenario and identify its limitationsHow Hill Climbing Works

Hill Climbing is a local search algorithm that starts with an initial solution and continuously makes small improvements. It evaluates neighboring solutions and moves to the one that provides the greatest improvement. The process continues until no better neighboring solution is found.

Application in Traffic Signal Optimization

1. Start with an initial traffic signal schedule.
2. Slightly modify the green or red signal timings.
3. Evaluate whether the new timings reduce waiting time and congestion.
4. If the new solution is better, accept it.
5. Repeat the process until no further improvement is possible.

Example

Suppose the green light duration is **30 seconds**.

- Change it to **35 seconds**.
- If congestion decreases, keep the new timing.
- Otherwise, test another nearby timing such as **25 seconds**.
- Continue searching for improvements.

Limitations of Hill Climbing

1. Local Maximum

The algorithm may stop at a solution that is better than its immediate neighbors but is not the best overall solution.

2. Plateau

Sometimes many neighboring solutions have the same value. The algorithm cannot determine which direction to move and may stop searching.

3. Ridge

The best solution may require moving in several directions simultaneously, but Hill Climbing only makes one small change at a time. This slows progress or prevents reaching the optimal solution.

4. No Backtracking

Once Hill Climbing moves to a better solution, it never returns to previous solutions. If it makes a poor decision, it cannot recover.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

1. Local Maxima

A local maximum is a solution that is better than nearby solutions but is not the global optimum.

Real-World Example

An intersection operates efficiently after adjusting one traffic signal, but congestion remains severe across the city because neighboring intersections are not optimized.

2. Plateau

A plateau is a flat area where many neighboring solutions have the same evaluation value.

Real-World Example

Changing the green signal duration between **40 and 45 seconds** produces almost identical traffic flow. The algorithm cannot determine which timing is better and may stop searching.

3. Ridge

A ridge occurs when improvement requires multiple coordinated changes rather than a single adjustment.

Real-World Example

Reducing congestion may require simultaneously adjusting the signal timings of several connected intersections. Changing only one intersection at a time provides little or no improvement, making it difficult for Hill Climbing to find the best solution.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

A. Simulated Annealing

Working Principle

Simulated Annealing improves upon Hill Climbing by occasionally accepting worse solutions during the early stages of the search. This allows the algorithm to escape local maxima and continue exploring the search space.

Advantages

- Escapes local maxima.
- Handles plateaus more effectively.
- Explores a larger search space.
- Suitable for dynamic optimization problems.

Traffic Example

If increasing a green signal duration temporarily worsens congestion, Simulated Annealing may still accept this change because it could lead to a better overall solution after further adjustments.

B. Genetic Algorithm

Working Principle

A Genetic Algorithm is inspired by natural evolution. It maintains a population of possible solutions and improves them over generations using selection, crossover, and mutation.

Steps

1. Generate multiple traffic signal timing solutions.
2. Evaluate each solution using a fitness function.
3. Select the best-performing solutions.
4. Combine them using crossover.
5. Introduce random mutations to create diversity.
6. Repeat until an optimal solution is found.

Advantages of Genetic Algorithm

- Searches many solutions simultaneously.
- Avoids local maxima.
- Handles very large search spaces.
- Easily adapts to changing traffic conditions.
- Produces high-quality solutions for complex optimization problems.

Traffic Example

Different signal timing plans are tested simultaneously across many intersections. The best plans are combined to produce improved traffic management strategies over successive generations.

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Recommended Algorithm: Genetic Algorithm

The Genetic Algorithm is the most suitable approach for smart city traffic optimization because it effectively handles large, dynamic, and complex search spaces.

Justification

1. Excellent Scalability

A city may contain hundreds of intersections, each with multiple signal timing options. The Genetic Algorithm can efficiently search this enormous solution space by evaluating many candidate solutions at the same time.

2. High Adaptability

Traffic patterns change throughout the day due to peak hours, accidents, road construction, or public events. Genetic Algorithms continuously evolve better solutions, making them highly adaptable to changing conditions.

3. Avoids Local Optima

Unlike Hill Climbing, which may become trapped in local maxima, Genetic Algorithms use crossover and mutation to explore diverse solutions and increase the likelihood of finding the global optimum.

4. Better Overall Optimization

The algorithm considers the entire traffic network rather than optimizing a single intersection. This leads to reduced waiting time, lower fuel consumption, and improved traffic flow across the city.

5. Suitable for Real-Time Smart Cities

Modern computing systems can run Genetic Algorithms efficiently, allowing traffic signal timings to be updated periodically based on real-time traffic data.

Conclusion

Traffic signal optimization in a smart city is a large-scale optimization problem where traditional search methods are impractical due to the enormous search space and continuously changing traffic conditions. Hill Climbing can quickly improve a solution but often gets trapped in local maxima, plateaus, or ridges. Simulated Annealing overcomes some of these limitations by allowing occasional worse moves, while the Genetic Algorithm provides the best overall performance by exploring multiple solutions simultaneously and adapting to dynamic traffic patterns. Therefore, **the Genetic Algorithm is the most suitable approach because it offers excellent scalability, adaptability, and the ability to find high-quality solutions for real-time traffic management.**

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

a) **Explain why this problem requires an online search agent instead of offline search**

b) **Analyze the challenges of partial observability and dynamic environments**

c) **Describe how the rover builds and updates its internal model**

d) **Compare online search with uninformed and informed search strategies**

e) **Justify the most suitable approach considering uncertainty, adaptability, and safety**

ANSWER:

3. Autonomous Rover on Mars (Online Search Agent)
Scenario Understanding

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. The rover must navigate safely while avoiding hazards such as rocks, craters, and steep slopes. Since it has limited sensing capability and no complete map of the environment, it must make decisions based on the information available at the current moment. As it moves, the rover continuously updates its knowledge about the surroundings. Therefore, this is a real-time decision-making problem that requires an intelligent search approach.

i. Explain why this problem requires an online search agent instead of offline search
Online Search Agent

An **online search agent** makes decisions while interacting with the environment. It does not require complete knowledge of the terrain before starting the search. The rover explores the environment, gathers new information through its sensors, and updates its path as it moves.

Why Online Search is Required

- The rover does not have a complete map of Mars.
- New obstacles such as rocks or craters are discovered only when the rover gets closer.
- The rover must make decisions immediately using the available information.
- It continuously updates its path based on newly collected data.
- It is suitable for unknown and changing environments.

Example

The rover plans to move forward, but its sensors detect a large crater. It immediately changes direction and selects a safer route without stopping the mission.

Why Offline Search is Not Suitable

Offline search requires complete knowledge of the environment before planning a path.

Limitations

- A complete map of Mars is unavailable.
- Hidden obstacles cannot be predicted in advance.
- The environment changes as new information is discovered.
- The planned path may become unsafe during exploration.

Therefore, offline search is not practical for Mars exploration.

ii. Analyze the challenges of partial observability and dynamic environments
1. Partial Observability

Partial observability means the rover can only observe a limited area around its current position.

Challenges

- Unknown obstacles may appear suddenly.
- The rover cannot see distant hazards.
- Sensor range is limited.
- Decisions must be made with incomplete information.

Example

The rover sees only nearby rocks. A deep crater located beyond its sensor range becomes visible only after moving closer.

2. Dynamic Environment

A dynamic environment changes while the rover is operating.

Challenges

- Dust storms may reduce visibility.
- Loose soil may affect movement.
- Newly detected hazards require immediate path changes.
- Communication delays with Earth prevent immediate human control.

Example

A dust storm reduces sensor accuracy, forcing the rover to slow down and choose a safer route.

iii. Describe how the rover builds and updates its internal model

The rover maintains an **internal model** of the environment based on sensor data and previous exploration.

Step 1: Observe the Environment

The rover uses cameras, LiDAR, radar, and other sensors to detect nearby objects and terrain features.

Step 2: Build an Internal Map

It records information such as:

- Safe paths
- Rocks
- Craters
- Slopes
- Sample locations

This information forms a partial map of the explored area.

Step 3: Update the Model

Whenever new information is received:

- Newly discovered obstacles are added.
- Unsafe routes are marked.
- Safe paths are updated.
- Navigation plans are recalculated.

Step 4: Make New Decisions

Using the updated internal model, the rover selects the safest and most efficient path to continue its mission.

Example

Initially, the rover assumes a path is clear. After detecting large rocks, it updates its map, marks the route as unsafe, and chooses an alternative path.

iv. Compare online search with uninformed and informed search strategies

Feature	Online Search	Uninformed Search	Informed Search
Environment Knowledge	Partial	Complete	Complete or estimated
Uses Heuristic	Yes (if available)	No	Yes
Suitable for Unknown Environment	Yes	No	Limited
Updates During Search	Yes	No	Usually No
Adaptability	High	Low	Moderate
Decision Making	Real-time	Pre-planned	Guided by heuristic

Online Search

- Learns while exploring.
- Continuously updates its path.
- Best suited for unknown environments.

Uninformed Search

- Does not use heuristic information.
- Explores many unnecessary paths.
- Inefficient for large unknown environments.

Informed Search

- Uses heuristic information to guide the search.
- Performs better than uninformed search when the environment is known.
- Less effective if the environment changes frequently.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety
Recommended Approach: Online Search Agent

An Online Search Agent is the most suitable approach for the Mars rover because it can make intelligent decisions in an unknown and changing environment.

Justification
1. Handles Uncertainty

The rover does not know the complete terrain before starting its mission. Online search allows it to explore safely while collecting new information.

2. High Adaptability

The rover continuously updates its internal map whenever new obstacles or hazards are detected. This allows it to quickly change its route without restarting the search.

3. Improved Safety

The rover avoids dangerous areas such as craters, steep slopes, and large rocks by using real-time sensor data. This reduces the risk of damage and mission failure.

4. Efficient Decision Making

Instead of planning the entire journey in advance, the rover makes decisions step by step. This enables it to respond immediately to unexpected situations.

5. Suitable for Real-Time Exploration

Since communication with Earth involves significant delays, the rover must operate independently. An Online Search Agent enables autonomous navigation and supports successful exploration in remote environments.

Conclusion

The Mars rover operates in an unknown and partially observable environment where complete information is unavailable before exploration. Offline search methods are not suitable because they require a complete map. An **Online Search Agent** continuously observes the environment, updates its internal model, and makes real-time navigation decisions. Compared with uninformed and informed search strategies, online search provides greater adaptability, better handling of uncertainty, and improved safety. Therefore, **an Online Search Agent is the most suitable approach for autonomous Mars exploration because it enables safe, efficient, and intelligent decision-making in dynamic environments.**

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. **Clearly define variables, domains, and constraints with explanation**

ii. **Explain how backtracking search works in this scenario**

iii. **Discuss how constraint propagation (e.g., forward checking) improves efficiency**

iv. **Analyze real-world challenges and justify the best strategy for scalability**

ANSWER:

4. University Exam Timetable Generation (Constraint Satisfaction Problem - CSP)

Scenario Understanding

A university wants to develop an AI system to automatically generate exam timetables. The timetable must satisfy several constraints such as preventing students from having overlapping exams, assigning available exam halls, considering faculty availability, scheduling certain subjects before others, and providing special accommodations for some students. As the number of courses and students increases, the problem becomes more complex. Therefore, this problem is best modeled as a **Constraint Satisfaction Problem (CSP)**.

Problem Formulation as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) consists of three main components:

- **Variables** – The elements that need values.
- **Domains** – The possible values each variable can take.
- **Constraints** – Rules that must be satisfied.

i. Clearly define variables, domains, and constraints with explanation

1. Variables

Variables represent the exams that need to be scheduled.

Examples

- Exam of Mathematics
- Exam of Physics
- Exam of Chemistry
- Exam of Computer Science
- Exam of English

Each subject is considered as one variable whose exam time and hall must be assigned.

2. Domains

The domain is the set of possible values for each variable.

Examples

Time Slots

- Monday Morning
- Monday Afternoon
- Tuesday Morning
- Tuesday Afternoon
- Wednesday Morning

Exam Halls

- Hall A
- Hall B
- Hall C
- Hall D

Each exam can be assigned to any available time slot and exam hall, provided all constraints are satisfied.

3. Constraints

Constraints are the rules that the timetable must satisfy.

a) No Student Overlap

A student enrolled in multiple subjects must not have two exams at the same time.

Example

If a student studies Mathematics and Physics, both exams cannot be scheduled in the same time slot.

b) Limited Exam Halls

The number of exams conducted at the same time cannot exceed the available exam halls.

Example

If only four halls are available, only four exams can be scheduled simultaneously.

c) Subject Order Constraint

Some subjects must be conducted before others.

Example

Programming Fundamentals must be scheduled before Advanced Programming.

d) Faculty Availability

The assigned faculty member or invigilator must be available during the examination.

Example

If Professor A is unavailable on Tuesday, exams requiring Professor A cannot be scheduled on that day.

e) Special Accommodation

Students requiring special facilities must be assigned suitable examination halls.

Example

A student with a disability should be assigned an accessible exam hall.

ii. Explain how backtracking search works in this scenario
Backtracking Search

Backtracking is a systematic search algorithm used to find a valid solution by assigning values one variable at a time. If a constraint is violated, the algorithm goes back (backtracks) and tries another assignment.

Working Steps
Step 1

Select the first exam.

Assign a time slot and exam hall.

Step 2

Select the next exam.

Assign its time slot and hall.

Check whether all constraints are satisfied.

Step 3

Continue assigning values to the remaining exams.

Step 4

If a constraint is violated, return to the previous assignment and choose another available value.

Step 5

Repeat the process until every exam has been assigned successfully.

Example

Suppose:

- Mathematics → Monday Morning
- Physics → Monday Morning

A student is enrolled in both subjects.

This violates the **No Student Overlap** constraint.

The algorithm backtracks and changes Physics to **Monday Afternoon**, making the timetable valid.

Advantages of Backtracking

- Guarantees a valid solution if one exists.
- Checks constraints during assignment.
- Eliminates invalid schedules early.
- Suitable for CSP problems.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
Constraint Propagation

Constraint propagation reduces the number of possible values before making future assignments.

One common technique is **Forward Checking**.

Forward Checking

Whenever a value is assigned to a variable, forward checking removes inconsistent values from the domains of the remaining variables.

Working Example

Suppose:

Mathematics is scheduled on **Monday Morning**.

Any subject having students in common with Mathematics can no longer use **Monday Morning**.

This prevents future conflicts before they occur.

Advantages of Forward Checking

1. **Reduces Search Space**

Many invalid choices are removed before they are explored.

2. **Detects Conflicts Early**

If a subject has no remaining valid time slots, the algorithm immediately backtracks.

3. **Improves Efficiency**

Less unnecessary searching is performed, reducing execution time.

4. **Faster Timetable Generation**

The algorithm reaches a valid timetable more quickly than simple backtracking.

iv. Analyze real-world challenges and justify the best strategy for scalability

Real-World Challenges

1. **Large Number of Courses**

Universities may offer hundreds of courses, making scheduling more complex.

2. **Large Student Population**

Many students register for multiple subjects, increasing the possibility of timetable conflicts.

3. **Limited Resources**

The number of available exam halls and invigilators is limited.

4. **Faculty Constraints**

Faculty members may not be available during every time slot.

5. **Special Requirements**

Some students require accessible halls or additional examination time.

6. **Last-Minute Changes**

Unexpected events such as hall maintenance, faculty absence, or emergency holidays may require the timetable to be modified.

Best Strategy for Scalability

Recommended Strategy: Backtracking Search with Forward Checking (Constraint Propagation)

This combination is the most suitable approach for large university exam timetabling problems.

Justification

1. **Efficient Constraint Handling**

Backtracking ensures that all timetable constraints are satisfied before finalizing the schedule.

2. Faster Search

Forward checking removes invalid options early, reducing unnecessary computations.

3. Scalable for Large Universities

Even when the number of courses and students increases significantly, constraint propagation greatly reduces the search space and improves performance.

4. Produces Valid Timetables

The algorithm ensures that no student has overlapping exams, resources are allocated correctly, and all scheduling rules are satisfied.

5. Adaptable

If new constraints arise, such as a faculty member becoming unavailable or an exam hall closing, the timetable can be updated by reapplying the CSP algorithm.

Conclusion

The university exam scheduling problem is a classic **Constraint Satisfaction Problem (CSP)** because it involves assigning exams to time slots and halls while satisfying multiple constraints. **Backtracking Search** systematically explores possible schedules and finds a valid solution, while **Forward Checking (Constraint Propagation)** improves efficiency by eliminating invalid choices early. Together, these techniques reduce computation time, handle large-scale scheduling problems effectively, and generate conflict-free exam timetables. Therefore, **Backtracking Search combined with Forward Checking is the most suitable strategy because it provides scalability, efficiency, and reliable timetable generation for universities.**

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. **Explain how the Minimax algorithm models this problem**

ii. **Analyze the computational challenges of large game trees**

iii. **Explain how Alpha-Beta pruning improves efficiency with detailed reasoning**

iv. **Design an evaluation function and justify the factors considered**

v. **Recommend a strategy for balancing optimality and real-time performance**

ANSWER:

5. Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Scenario Understanding

A gaming company is developing an AI opponent for a real-time strategy game. The AI competes against human players and must make intelligent decisions within a short time. Every move creates many possible future game states, resulting in a very large decision tree. The AI must choose the best move while balancing optimality and computation time. Since evaluating every possible move is computationally expensive, efficient search techniques are required.

i. Explain how the Minimax algorithm models this problem

Minimax Algorithm

The **Minimax algorithm** is an adversarial search algorithm used in two-player games where one player tries to maximize the score while the other tries to minimize it.

In this scenario:

- **AI Player (MAX):** Tries to choose the move that gives the highest advantage.
- **Human Player (MIN):** Tries to choose the move that reduces the AI's advantage.

The AI builds a game tree by exploring possible future moves.

Working Steps

Step 1

Generate all possible moves for the AI.

Step 2

For each AI move, generate all possible responses by the human player.

Step 3

Continue expanding the game tree up to a certain depth.

Step 4

Evaluate the final game states using an evaluation function.

Step 5

The AI selects the move that maximizes its minimum possible gain (Minimax principle).

Example

Suppose the AI has three possible moves:

- Move A → Score = 6
- Move B → Score = 8
- Move C → Score = 5

The human player always tries to reduce the AI's score.

After considering the opponent's best responses, the AI chooses the move that provides the **best worst-case outcome**.

Advantages of Minimax

- Makes intelligent decisions.
- Considers the opponent's possible moves.
- Produces optimal decisions when the complete game tree is explored.
- Suitable for strategic games.

Disadvantages

- Requires large computation.
- Search time increases rapidly with tree depth.
- Not suitable alone for very large real-time games.

ii. Analyze the computational challenges of large game trees

Real-time strategy games have an enormous number of possible game states.

Challenges

1. Large Branching Factor

Each game state may have many possible moves.

Example

If each position has 20 possible moves and the search depth is 5:

Total nodes = $\mathbf{20^5 = 3{,}200{,}000}$

This requires significant computation.

2. High Memory Usage

The AI must store many game states while searching.

As the tree grows deeper, memory requirements increase.

3. Time Constraints

The AI often has only a few milliseconds to make a decision.

Searching the complete game tree is impossible in real-time.

4. Complex Evaluation

Each game state includes multiple factors such as:

- Resources
- Units
- Buildings
- Enemy positions
- Map control

Evaluating every state increases computation time.

5. Dynamic Game Environment

The opponent's actions continuously change the game state.

The AI must frequently recalculate its decisions.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm.

It eliminates branches of the game tree that cannot affect the final decision.

As a result, the AI evaluates fewer nodes while still producing the same optimal move.

Working Principle

Two values are maintained:

Alpha (α)

The best score that the **MAX (AI)** player can guarantee so far.

Beta (β)

The best score that the **MIN (Human)** player can guarantee so far.

Pruning Condition

Whenever:

α ≥ β

the remaining branches are ignored because they cannot improve the final decision.

Example

Suppose:

Branch A

AI can achieve score = **8**

So,

Alpha = 8

Branch B

The opponent can force the score down to **5**.

Since

Beta = 5

Now,

Alpha (8) ≥ Beta (5)

The remaining nodes in Branch B are **pruned** because they cannot produce a better outcome.

The AI immediately moves to another branch.

Advantages of Alpha-Beta Pruning

1. Reduces Search Time

Many unnecessary nodes are ignored.

2. Searches Deeper

The saved computation allows the AI to explore more levels within the same time limit.

3. Produces the Same Optimal Result

Although fewer nodes are evaluated, the final decision remains the same as the standard Minimax algorithm.

4. Suitable for Real-Time Games

The AI responds faster, making it practical for games with strict time limits.

An **evaluation function** assigns a score to each game state so that the AI can compare different positions.

Example Evaluation Function

Evaluation Score =

- (Unit Strength × 5)
  -
  - (Resources × 3)
  -
  - (Map Control × 4)
  -
  - (Building Strength × 2)
- – (Enemy Strength × 5)
- – (Units Lost × 3)

Factors Considered

1. Unit Strength

A larger and stronger army increases the score because it improves the AI's chances of winning.

2. Resources

More resources allow the AI to build units and upgrade technology.

3. Map Control

Controlling important areas of the map provides strategic advantages.

4. Building Strength

Protecting important buildings ensures continued unit production and defense.

5. Enemy Strength

A powerful opponent reduces the evaluation score because it increases the difficulty of winning.

6. Units Lost

Losing valuable units weakens the AI's position, so the score is reduced.

v. Recommend a strategy for balancing optimality and real-time performance
Recommended Strategy: Minimax with Alpha-Beta Pruning

The best approach is to use **Minimax combined with Alpha-Beta Pruning**.

Justification

1. Optimal Decision Making

Minimax analyzes both the AI's and the opponent's possible moves, leading to strong strategic decisions.

2. **Faster Computation**

Alpha-Beta Pruning eliminates unnecessary branches, reducing the number of game states that need to be evaluated.

3. **Suitable for Real-Time Games**

The AI can make decisions within strict time limits while maintaining good performance.

4. **Better Resource Utilization**

By pruning irrelevant branches, the algorithm uses less memory and processing power.

5. **Improved Scalability**

The AI can handle larger game trees and more complex game scenarios without sacrificing decision quality.

Conclusion

In real-time strategy games, the AI must make intelligent decisions quickly while considering the opponent's possible actions. The **Minimax algorithm** models the game as a competition between a maximizing player (AI) and a minimizing player (human opponent). However, large game trees create computational challenges due to the vast number of possible moves. **Alpha-Beta Pruning** improves efficiency by removing branches that cannot influence the final decision, allowing the AI to search deeper within the same time limit. An evaluation function based on unit strength, resources, map control, buildings, enemy strength, and unit losses helps the AI assess game states effectively. Therefore, **Minimax combined with Alpha-Beta Pruning is the most suitable strategy because it provides an excellent balance between optimal decision-making and real-time performance.**

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrate s clear and complete understandi n g of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand